FlashNorm: Fast Normalization for Transformers

Nils Graef[*], **Filip Makraduli**, **Andrew Wasielewski**, **Matthew Clapp**
OpenMachine

Abstract

Normalization layers are ubiquitous in large language models (LLMs) yet represent a compute bottleneck: on hardware with distinct vector and matrix execution units, the RMS calculation blocks the subsequent matrix multiplication, preventing parallel execution. We present **FlashNorm**, an *exact* reformulation of RMSNorm followed by a linear layer that (i) eliminates the normalization weights by folding them into the subsequent linear layer, and (ii) defers the scalar RMS normalization to the output of the matrix multiplication, enabling the two operations to execute in parallel. FlashNorm is mathematically identical to the original computation—it introduces no approximation and requires no retraining. The same technique extends to LayerNorm, Dynamic Tanh (DyT), feed-forward networks with GLU variants, and RoPE-based attention. On an NVIDIA T4 GPU, FlashNorm achieves **33–35%** lower latency on the norm-then-project operation in the compute-bound (prefill) regime at SmolLM2-135M scale, and **12–14%** at Llama-7B scale. We verify zero-loss weight folding on SmolLM2-135M, Llama-3.2-1B, and Llama-3.1-8B. Beyond inference speed, FlashNorm simplifies model implementations by reducing parameter tensor count, analogous to the simplification achieved by PaLM's removal of bias-parameters from all linear layers. Watch our explainer video [1] and see [2] for code.

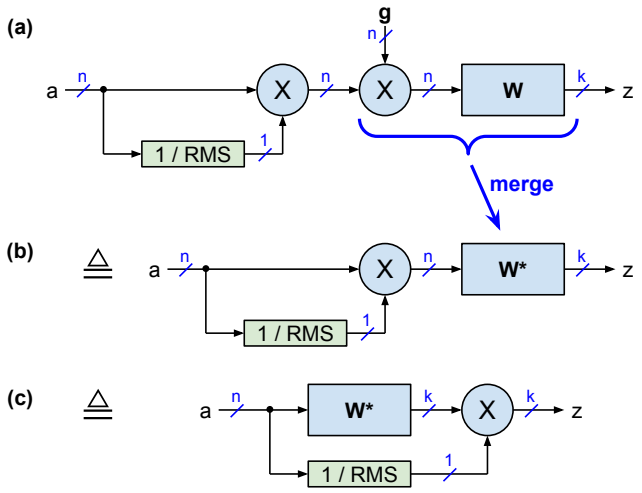

Figure 1: Mathematically identical implementations of RMSNorm followed by a linear layer. (a) Standard: normalization weights $\mathbf{g}$ are applied element-wise before $\mathbf{W}$. (b) **Weightless**: $\mathbf{g}$ is folded into $\mathbf{W}^* = \mathrm{diag}(\mathbf{g})\mathbf{W}$. (c) **FlashNorm**: normalization is deferred to the output, enabling matrix multiplication ($\mathbf{W}^*$) and RMS calculation to execute in parallel. $\triangleq$ denotes mathematical identity.

[*] info@openmachine.ai

1 Introduction

Normalization is indispensable to the training stability and generalization of modern LLMs. RMSNorm [3], used by Llama [4], Gemma [5], Mistral [6], and OLMo 2 [7], normalizes each activation vector by its root mean square (RMS) before scaling by learned weights. In the standard transformer network [8], RMSNorm is followed immediately by a linear (projection) layer—in both the attention and feed-forward sub-networks of every transformer block.

This sequential structure conceals a hardware inefficiency. On processors with dedicated vector units (for element-wise ops) and matrix units (for matrix multiplication), the RMS value must be fully computed *before* the linear layer can begin, because the normalized activations are its input. The matrix unit therefore sits idle during the RMS calculation, a bottleneck that compounds across all transformer layers and inference calls.

We address this bottleneck with **FlashNorm**, which makes two mathematically equivalent transformations to the RMSNorm-then-project pipeline shown in Fig. 1(a):

1. **Weightless normalization.** The per-channel normalization weights $\mathbf{g}$ are absorbed into the weight matrix of the following linear layer, eliminating them as a separate parameter tensor as illustrated in Fig. 1(b).
2. **Deferred normalization.** Because scaling by $1/\mathrm{RMS}(\mathbf{a})$ commutes with matrix multiplication, the scalar normalization is moved to the *output* of the linear layer as shown in Fig. 1(c). This allows the matrix multiplication and the RMS calculation to execute in parallel on separate hardware units.

The name is inspired by FlashAttention [9], where the softmax denominator is similarly deferred past a matrix multiplication to enable parallel processing of keys and values.

FlashNorm requires no retraining, and weight folding can be applied post-hoc to any pretrained checkpoint. We further show how the same ideas extend to Layer Normalization [10], DyT [11], GLU-variant FFNs [12], and RoPE-based attention [13].

Contributions.

- We formally characterize the commutativity of RMSNorm with matrix multiplication (Proposition 2) and derive the exact conditions under which deferred normalization holds.
- We present FlashNorm as a unified framework covering RMSNorm, LayerNorm, DyT, GLU-FFNs, and attention with RoPE.
- We verify zero-loss weight-folding on SmolLM2-135M, Llama-3.2-1B, and Llama-3.1-8B across wikitext, MMLU, and HellaSwag (Table 1), and report operation-level speedups on T4, A100, and H100.
- And we identify the kernel-engineering work required to extend to compute-bound regimes and provide a reference design.

2 Background and Related Work

Normalization in transformers. Layer Normalization [10] applies mean centering followed by variance normalization with learned scale and bias. RMSNorm [3] drops mean centering and bias, reducing to pure RMS-based scaling; it has become the dominant normalization choice in recent open LLMs due to its simplicity and comparable empirical performance. Dynamic Tanh (DyT) [11] is a recently proposed drop-in replacement that eliminates the normalization computation entirely, replacing it with a parameterized $\tanh$; FlashNorm is complementary in that it targets the interaction between normalization and the subsequent linear layer.

Kernel fusion and operation reordering. Fusing small operations into a single GPU kernel is a standard optimization to reduce memory bandwidth and kernel-launch overhead. Mehta et al. [14] report that modifications to the layer normalization implementation significantly affect tokens-per-second throughput, attributing this to a lack of kernel fusion. FlashNorm reduces the number of kernel launches by eliminating the normalization weight application as a separate step, and additionally enables parallelism between the vector and matrix units that kernel fusion alone cannot achieve.

Weight folding and parameter elimination. The idea of absorbing one layer's parameters into a neighboring layer appears in several forms. Batch normalization folding into convolutional weights is standard practice in quantization-aware training and model deployment [15, 16] and `torch.compile` can perform it automatically [17]. OLMo 1 models use weightless LayerNorm, which they call *non-parametric* [7]. FlashNorm applies analogous reasoning to normalization weights of RMSNorm and LayerNorm.

FlashAttention. FlashAttention [9] is the closest conceptual predecessor: it defers the softmax normalization (division by $\sum \exp$) past the multiplication with the value matrix $\mathbf{V}$, enabling online computation and improving memory efficiency. FlashNorm applies the same reordering insight to the simpler setting of RMSNorm and a dense linear layer.

3 FlashNorm

We now develop FlashNorm formally. All propositions assume bias-free linear layers unless stated otherwise; Section 3.3 addresses the biased case.

3.1 Weightless Normalization

Let $\mathbf{a} \in \mathbb{R}^n$ be an activation vector. RMSNorm computes

$$\mathrm{RMSNorm}(\mathbf{a})_i = \frac{a_i}{\mathrm{RMS}(\mathbf{a})} \cdot g_i, \qquad \mathrm{RMS}(\mathbf{a}) = \sqrt{\frac{1}{n}\sum_{i=1}^{n} a_i^2},$$

where $\mathbf{g} \in \mathbb{R}^n$ are learned normalization weights. In transformer models, RMSNorm is followed by a linear layer $\mathbf{W} \in \mathbb{R}^{n\times k}$ as illustrated in Fig. 1(a):

$$\mathbf{z} = \mathrm{RMSNorm}(\mathbf{a})\,\mathbf{W}.$$

Proposition 1 (Weightless normalization)**.** *Let $\mathbf{W}^* \in \mathbb{R}^{n\times k}$ be defined by $W^*_{i,j} = g_i \cdot W_{i,j}$. Then $\mathbf{z} = \frac{\mathbf{a}}{\mathrm{RMS}(\mathbf{a})}\mathbf{W}^*$, i.e., the normalization weights $\mathbf{g}$ can be absorbed into $\mathbf{W}$ and need not be stored or applied separately.*

Proof. $z_j = \sum_i \frac{a_i}{\mathrm{RMS}(\mathbf{a})} g_i W_{i,j} = \sum_i \frac{a_i}{\mathrm{RMS}(\mathbf{a})} W^*_{i,j}$. □

This proposition holds for linear layers with or without a bias term at the output. The practical implication is that the normalization weight vector $\mathbf{g}$ becomes a redundant parameter: it can be folded into $\mathbf{W}$ once at load time, saving memory, reducing the parameter count, and eliminating one vector-multiply kernel call per forward pass.

3.2 Deferred Normalization

Proposition 2 (Commutativity of RMS scaling with matrix multiplication)**.** *Let $\mathbf{W}^*$ be as in Proposition 1 and let the linear layer be bias-free. Then*

$$\mathbf{z} = \frac{\mathbf{a}}{\mathrm{RMS}(\mathbf{a})}\mathbf{W}^* = (\mathbf{a}\,\mathbf{W}^*) \cdot \frac{1}{\mathrm{RMS}(\mathbf{a})}.$$

That is, the scalar normalization $1/\mathrm{RMS}(\mathbf{a})$ can be applied after *the matrix multiplication.*

Proof. Scalar multiplication commutes with matrix multiplication: $(\alpha\mathbf{a})\mathbf{W}^* = (\mathbf{a}\mathbf{W}^*)\alpha$ for any scalar α. Setting $\alpha = 1/\mathrm{RMS}(\mathbf{a})$ gives the result. □

Remark 1 (Bias case)**.** *If the linear layer has a bias $\mathbf{c} \in \mathbb{R}^k$, deferred normalization does not hold directly because $(\mathbf{a}\mathbf{W}^* + \mathbf{c})/\mathrm{RMS}(\mathbf{a}) \neq \mathbf{a}\mathbf{W}^*/\mathrm{RMS}(\mathbf{a}) + \mathbf{c}$. In this case, Proposition 1 (weightless normalization) still applies, but the normalization must precede the bias addition.*

Corollary 1 (Parallel execution)**.** *Under the conditions of Proposition 2, computing $\mathbf{a}\mathbf{W}^*$ and $\mathrm{RMS}(\mathbf{a})$ are* independent *computations that may proceed in parallel. The final result requires only a single vector-scalar multiply after both complete.*

Figure 1 illustrates the three mathematically equivalent implementations: (a) standard RMSNorm-then-project, (b) weightless normalization, and (c) the full FlashNorm with deferred normalization.

3.3 Support for Normalization Bias

LayerNorm [10] and DyT [11] include an additive bias $\boldsymbol{\beta}$ after scaling by $\mathbf{g}$. Fig. 2 illustrates how this bias can be moved to the output of the following linear layer, resulting in a modified bias $\mathbf{c}^* = \mathbf{c} + \boldsymbol{\beta}\mathbf{W}$, see Fig. 2(b). After this elimination of $\boldsymbol{\beta}$, the normalization weights $\mathbf{g}$ can be folded into the linear layer as per Proposition 1. Note that deferred normalization (Proposition 2) does not apply when a bias is present; the scalar $1/\mathrm{RMS}(\mathbf{a})$ must be applied before adding $\mathbf{c}^*$, which still allows for parallel execution of matrix multiplication and RMS calculation.

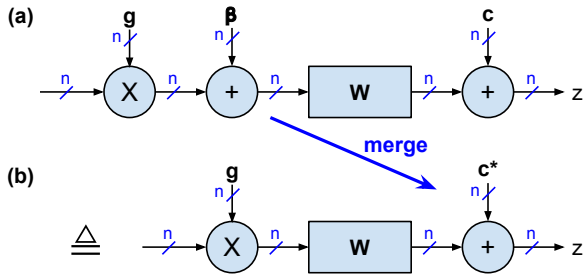

Figure 2: Elimination of bias $\boldsymbol{\beta}$: (a) Before elimination with $\boldsymbol{\beta}$ between normalization weights $\mathbf{g}$ and linear layer. (b) Optimized version with new bias term $\mathbf{c}^* = \mathbf{c} + \boldsymbol{\beta}\mathbf{W}$ at the output.

3.4 Folding Mean Centering into a Preceding Linear Layer

LayerNorm applies mean centering before RMSNorm. If mean centering is preceded by a linear layer with weights $\mathbf{V} \in \mathbb{R}^{n \times n}$, the centering can be absorbed into $\mathbf{V}$ without retraining.

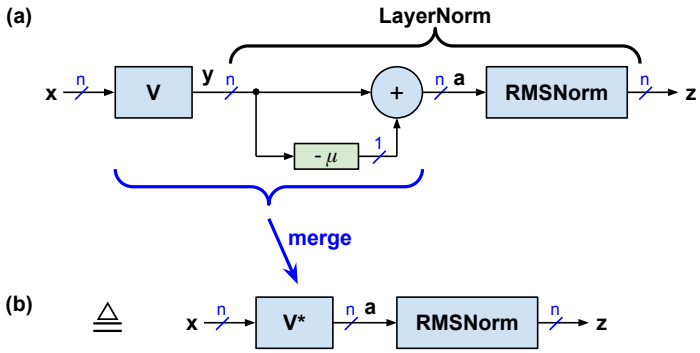

Figure 3: Elimination of mean centering: (a) Original weight matrix $\mathbf{V}$ followed by mean centering. (b) Optimized version where the mean centering is merged into the modified weight matrix $\mathbf{V}^*$.

The mean μ is calculated from the linear layer outputs y_j as $\mu = \frac{1}{n}\sum_j y_j$ where $y_j = \sum_i x_i V_{i,j}$. Let $s_i = \sum_j V_{i,j}$ be the row sum of row i, we can then calculate μ directly from the inputs x_i as

$$\mu = \frac{1}{n}\sum_{j=1}^{n}\sum_{i=1}^{n} x_i V_{i,j} = \frac{1}{n}\sum_{i=1}^{n} x_i \left[\sum_{j=1}^{n} V_{i,j}\right] = \frac{1}{n}\sum_{i=1}^{n} x_i s_i.$$

The mean-centered outputs are now

$$a_j = y_j - \mu = \sum_{i=1}^{n} x_i V_{i,j} - \frac{1}{n}\sum_{i=1}^{n} x_i s_i = \sum_{i=1}^{n} x_i \underbrace{\left(V_{i,j} - \frac{s_i}{n}\right)}_{V^*_{i,j}},$$

so the modified weights $V^*_{i,j} = V_{i,j} - s_i/n$ absorb the mean centering entirely, see Fig. 3(b). This also provides a lossless recipe for converting a LayerNorm model to RMSNorm without any retraining. Fig. 3(a) shows the weight matrix $\mathbf{V}$ followed by mean centering and RMSNorm.

4 Extensions

4.1 FlashNorm for FFNs with ReLU

For bias-free FFNs with ReLU, the scale-invariance $\mathrm{ReLU}(s \cdot \mathbf{a}) = s \cdot \mathrm{ReLU}(\mathbf{a})$ for $s \geq 0$ [18] allows the RMS normalization to be deferred all the way to the FFN output, past both the up-projection and the ReLU nonlinearity as illustrated in Fig. 4(b). If the up-projection expands from n to f channels, this saves $f - n$ multiply operations compared to applying the scaling at the FFN input.

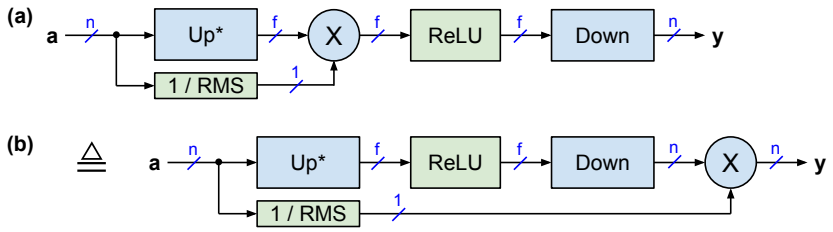

Figure 4: FFN with ReLU and preceding FlashNorm: (a) unoptimized version; (b) optimized version where the normalization is deferred to the output of the FFN. Up and Down denote the linear layers for up and down projections.

4.2 FlashNorm for FFNs with GLU Variants

For GLU-variant FFNs [12], the FlashNorm at the FFN input requires scaling at both the Gate and Up projection outputs, see Fig. 5(a). One of these scaling operations can be deferred to the FFN output, saving $f - n$ multiply operations as shown in Fig. 5(b). See appendix for ReGLU and Bilinear GLU variants.

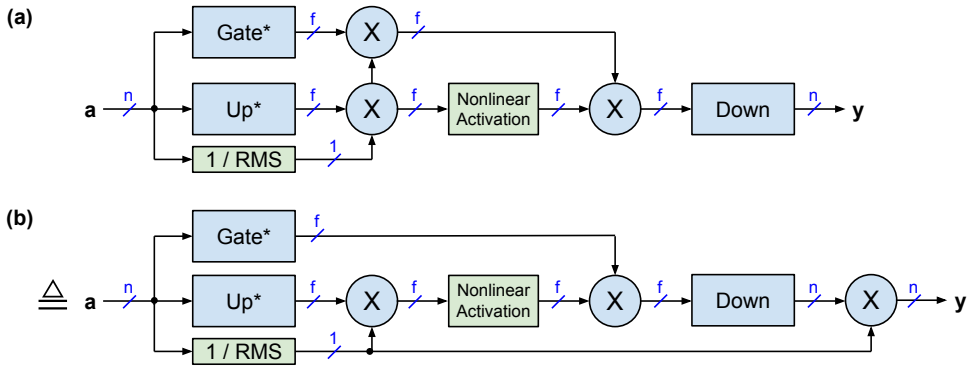

Figure 5: FFN with GLU variant and preceding FlashNorm: (a) unoptimized version; (b) optimized version with fewer scaling multipliers.

4.3 FlashNorm for Attention with RoPE

For the query ($\mathbf{Q}^*$) and key ($\mathbf{K}^*$) projections in RoPE-based attention [13], the $1/\mathrm{RMS}(\mathbf{a})$ factor can be fused with the precomputed RoPE cosine and sine tables, which are shared across all attention heads as shown in Fig. 6. Specifically, replacing $\cos(\cdot)$ with $\cos(\cdot)/\mathrm{RMS}(\mathbf{a})$ (and similarly for $\sin$) eliminates the per-head scaling entirely, saving $2hH - h$ multiply operations, where h is the head dimension and H is the number of heads. Additionally, the $1/\sqrt{h}$ scaled dot-product factor can be folded into the same scaling operation.

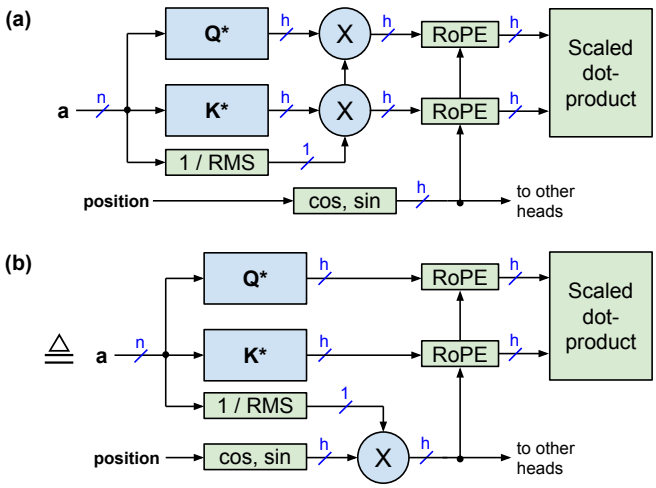

Figure 6: FlashNorm for scaled dot-product attention with RoPE: (a) unoptimized version; (b) optimized version where the normalization is fused with $\cos(\cdot)$ and $\sin(\cdot)$.

4.4 QK-Normalization

Gemma 3 [19], OLMo 2 [7], OpenELM [14] and other LLMs use per-head QK-normalization [20], which applies RMSNorm to queries and keys *after* the projection, preventing direct application of FlashNorm. Two optimizations are still available:

Eliminating the pre-projection RMS. By the scale-invariance of RMS, $\mathrm{RMS}(s \cdot \mathbf{a}) = s \cdot \mathrm{RMS}(\mathbf{a})$. When a FlashNorm RMS scaler s_a is followed immediately by a QK-norm RMS scaler s_c on the projected output as shown in Fig. 7 with activation vectors $\mathbf{a}, \mathbf{b}, \mathbf{c}$, the two scalers cancel: $s_a \cdot s_c = s_a \cdot (s_b/s_a) = s_b$ because $s_c = 1/\mathrm{RMS}(\mathbf{c}) = 1/\mathrm{RMS}(\mathbf{b} \cdot s_a) = 1/\left(s_a \cdot \mathrm{RMS}(\mathbf{b})\right) = s_b/s_a$. The pre-projection RMS calculation is therefore redundant and can be eliminated as illustrated in Fig. 7(b). The scale-invariance property of $\mathrm{RMS}(\vec{a})$ doesn't hold exactly true for RMS with epsilon (see appendix). This should not matter because the epsilon only makes an impact if the RMS (or energy) of the activation vector is very small, in which case the epsilon limits the up-scaling of this low-energy activation vector.

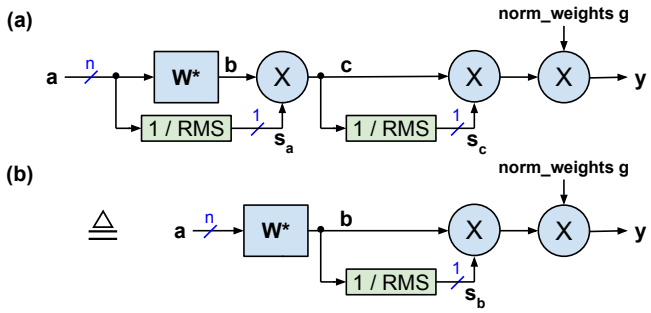

Figure 7: Linear layer with FlashNorm followed by a second normalization: (a) unoptimized version; (b) optimized version.

Fusing QK-norm weights with RoPE. If QK-norm weights are shared across all heads of a layer (as in Gemma 3 and OpenELM), they can be fused with the RoPE cosine and sine vectors, which are already shared across heads, see Fig. 8. The fused tables are computed once per layer, and the normalization weights require only a permutation to align with RoPE's interleaved layout: $\mathrm{permute}(\mathbf{g}) = (g_2, g_1, g_4, g_3, \ldots, g_h, g_{h-1})$.

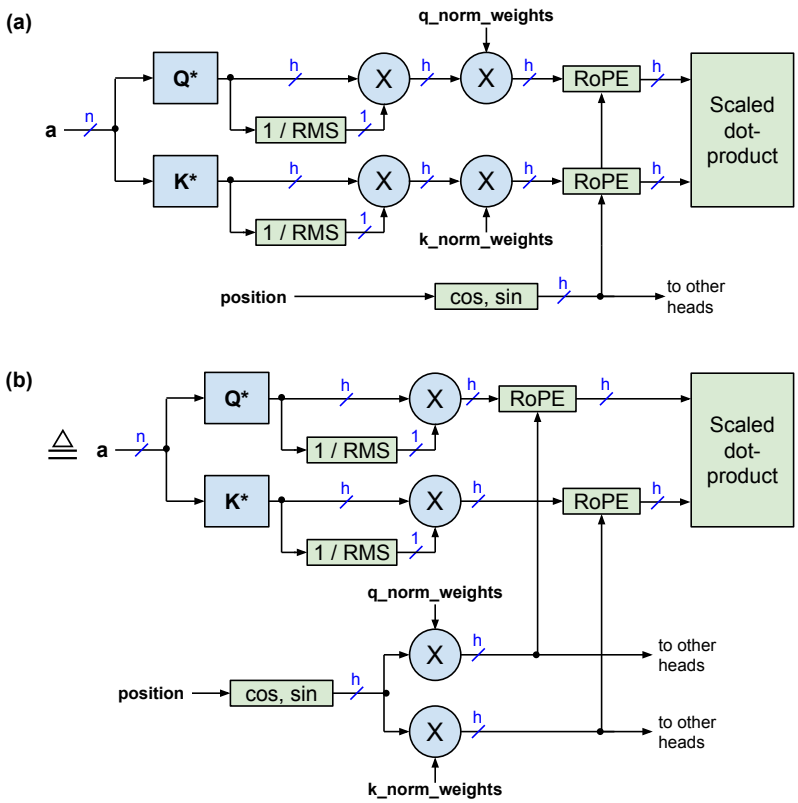

Figure 8: QK-norm with RoPE: (a) unoptimized version; (b) optimized version.

Moreover, several architectures employ partial RoPE or interleave RoPE and NoPE layers (e.g., Llama 4 [21]). For these models, we can fuse the Q and K normalization weights for the NoPE channels as $g_{\mathbf{QK},i} = g_{\mathbf{Q},i} * g_{\mathbf{K},i}$ for $i \in [\text{NoPE}]$.

5 Hardware Analysis

We analyze the compute bottleneck that FlashNorm eliminates. Consider a processor with one vector unit (throughput: m ops/cycle) and one matrix unit (throughput: m^2 ops/cycle) where m is the processor width. Specifically:

- Multiplying an n-vector with an $n \times n$ matrix takes n^2 MAD (multiply-add) operations, which takes n^2/m^2 matrix cycles.
- Computing $1/\text{RMS}(\mathbf{a})$ requires n MAD operations (for squaring and adding) plus 2 scalar ops (for $\sqrt{n/x}$), which takes n/m vector cycles if we ignore the 2 scalar ops.
- Scaling an n-vector by a scalar takes n/m vector cycles.

Fig. 9 shows timing diagrams for the example $n = 512$ and $m = 128$. Without FlashNorm (sequential execution), the matrix unit cannot begin until both the RMS and the scaling are complete, imposing an 8 cycle bubble as shown in Fig. 9(a). The subsequent vector-matrix multiply takes $(512)^2/(128)^2 = 16$ cycles; the total is 24 cycles.

With row-major ordering, the first rows of $\mathbf{W}^*$ can be processed while later rows of the RMS accumulation are still in flight, reducing the bubble to 5 cycles (total: 21 cycles), but the bottleneck persists, see Fig. 9(b).

With FlashNorm (deferred normalization), the vector-matrix multiply ($\mathbf{aW}^*$) and the RMS calculation proceed fully in parallel on separate units as shown in Fig. 9(c). The scaling at the end can be performed in parallel to the matrix unit if $\mathbf{W}^*$ is processed in column-major order. Total execution: 17 cycles—a **29% reduction** versus the sequential baseline.

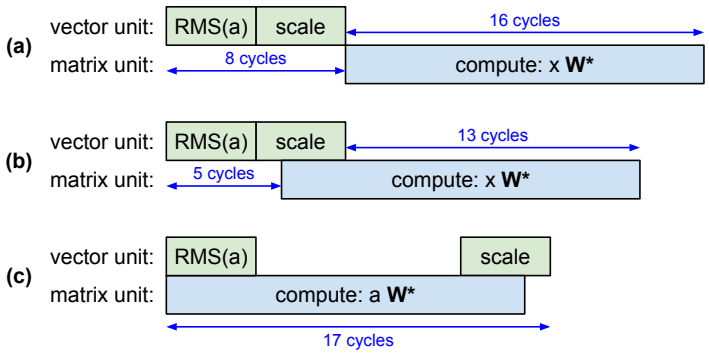

Figure 9: Timing diagrams for $n = 512, m = 128$. (a) Sequential: matrix unit idles 8 cycles. (b) Partial overlap with interleaved scaling and vector-matrix multiplication. (c) **FlashNorm**: vector-matrix multiply and RMS run fully in parallel; 17 cycles total, a 29% reduction over (a).

6 Experiments

Refer to [22, 2] for Python code that demonstrates the mathematical equivalency of the optimizations presented in this paper. A reproducible Colab notebook for the GPU benchmarks below is available in the `transformer-tricks` repository[2].

Scope. The mathematics of FlashNorm (Propositions 1 and 2) is exact in both directions. Applying FlashNorm post-hoc to a trained model is lossless: the folded model's outputs are bit-identical to the source under fp32 and within benchmark noise under fp16. Training a new model directly with a FlashNorm-shaped normalization-plus-linear block is equally valid: the FlashNorm parameterization (no explicit $\mathbf{g}$, RMS scalar applied after the matmul) represents the same function class as standard RMSNorm followed by a linear layer, so any function reachable under the standard parameterization is reachable under the FlashNorm parameterization. Experiments in this paper cover the post-hoc application; training-from-scratch studies are left to future work.

We verified lossless weight folding on SmolLM2-135M [23], Llama-3.2-1B, and Llama-3.1-8B [4] using `flashify_repo()` from the `transformer-tricks` package [2]. On SmolLM2-135M under HuggingFace Transformers at fp16, the original and weight-folded models produce identical logits (max difference = 0.0, cosine similarity = 1.0) and identical generated text; at Llama-3.2-1B and Llama-3.1-8B scales under fp32 inference, greedy generation is likewise bit-identical to the source. Table 1 reports source-vs-flashified deltas across three model scales and three standard benchmarks (wikitext word perplexity, MMLU 5-shot accuracy, HellaSwag zero-shot accuracy): every delta is within benchmark noise, empirically consistent with the mathematical exactness of Propositions 1 and 2. Three public compatibility checkpoints are available under `https://huggingface.co/open-machine` and load unmodified in stock HuggingFace Transformers and vLLM.

Table 1: Quality preservation across model scales and standard benchmarks, via `lm-evaluation-harness` with fp16 inference on an NVIDIA A100 (wikitext at 2048-token context, MMLU at 4096-token context for 5-shot prompts, HellaSwag at 2048-token context). Deltas between source and FlashNorm-folded checkpoints are within benchmark noise at every row, empirically consistent with the mathematical exactness of Propositions 1 and 2.

Model	Benchmark	Source	FlashNorm	Δ
SmolLM2-135M	wikitext word_ppl	23.1262	23.1279	+0.001689
Llama-3.2-1B	wikitext word_ppl	12.8837	12.8842	+0.000497
Llama-3.1-8B	wikitext word_ppl	8.0668	8.0672	+0.000382
Llama-3.1-8B	MMLU acc (5-shot)	0.6556	0.6562	+0.0006
Llama-3.1-8B	HellaSwag acc (0-shot)	0.6071	0.6072	+0.0001

[2] `https://github.com/OpenMachine-ai/transformer-tricks/blob/main/notebooks/flashNorm_gpu_benchmark.ipynb`

The FlashNorm transformation is mathematically exact (Propositions 1 and 2), and under fp32 inference produces bit-identical greedy generation across all three model scales. Under lossy inference precisions (fp16/bf16 in HuggingFace Transformers at larger scales, or vLLM's PagedAttention at any precision), precomputed merged weights can produce a one-token argmax divergence at tight decision points, which downstream greedy decoding then amplifies. The effect is more visible at larger model scales where argmax decisions are tighter. This is a general consequence of precomputing weight-folded tensors for lossy-inference kernels and is not specific to FlashNorm. A native fused `RMSNorm+QKV` kernel that defers $\mathbf{g}$ to runtime, rather than precomputing it into $\mathbf{W}^*$, eliminates the framework dependency and is the primary direction for ongoing work.

To validate the parallel execution described in Fig. 9, we benchmarked the norm-then-project operation on an NVIDIA T4 GPU (65 TFLOPS FP16, 320 GB/s). We compare sequential execution (RMSNorm $\rightarrow$ GEMM) against FlashNorm with adaptive dispatch: when the GEMM is compute-bound (many tokens), the GEMM runs on tensor cores and $1/\text{RMS}(\vec{a})$ runs on vector cores in parallel via separate CUDA streams; when memory-bound (few tokens), FlashNorm falls back to sequential execution to avoid stream overhead. Table 2 shows the results at SmolLM2-135M scale, and Table 3 at Llama-7B scale.

Table 2: Norm-then-project latency on T4 GPU at SmolLM2-135M scale (n=576, k=960). Below the compute-bound threshold, FlashNorm falls back to sequential execution (0% overhead).

Tokens	Sequential	FlashNorm	Speedup	Path
1	0.374 ms	0.374 ms	0.0%	sequential (fallback)
64	0.506 ms	0.506 ms	0.0%	sequential (fallback)
4096	0.704 ms	0.468 ms	+33.6%	parallel (overlap)
8192	0.929 ms	0.599 ms	+35.5%	parallel (overlap)

Table 3: Norm-then-project latency on T4 GPU at Llama-7B scale (n=4096, k=4096).

Tokens	Sequential	FlashNorm	Speedup	Path
1	0.641 ms	0.641 ms	0.0%	sequential (fallback)
64	0.307 ms	0.307 ms	0.0%	sequential (fallback)
256	0.577 ms	0.577 ms	0.0%	sequential (fallback)
1024	1.882 ms	1.654 ms	+12.1%	parallel (overlap)
4096	7.628 ms	6.570 ms	+13.9%	parallel (overlap)

FlashNorm uses adaptive dispatch: the parallel path is used only when the GEMM is compute-bound (token count exceeds a hardware-dependent threshold). For memory-bound workloads such as autoregressive decoding, FlashNorm falls back to sequential execution (0% overhead). This is because managing parallel CUDA streams introduces scheduling overhead that exceeds the time saved when the GEMM is small. Profiling individual operations within a SmolLM2-135M decoder layer at 4096 tokens confirms that the $1/\text{RMS}(\vec{a})$ calculation takes $\sim$0.08 ms per layer. Across 30 layers, the theoretical maximum savings from perfect overlap is $\sim$5 ms out of $\sim$60 ms total ($\sim$8%).

While the speedup on the norm-then-project operation itself is significant (33–35%), the end-to-end model speedup is modest because this operation is a fraction of total layer time. For comparison, we measured a throughput of 204 tokens per second for OpenELM-270M with 4-bit weight quantization using the MLX framework on an M1 MacBook Air. This throughput increases to only 225 tokens per second when we remove RMSNorm entirely, giving an upper bound of $\leq$ 10% for this model. Native integration into inference frameworks at the kernel level—where stream management overhead is negligible—would allow the 33–35% operation-level improvement to translate more directly into end-to-end gains, particularly for larger models with bigger GEMMs.

6.1 Prototype Fused Kernel on A100

We further validate FlashNorm on an NVIDIA A100 GPU (312 TFLOPS FP16, 2 TB/s HBM) by prototyping a fused Triton kernel that merges `RMSNorm` with the subsequent QKV projection into a single GPU kernel, benchmarked against two production-realistic baselines: PyTorch's fused `torch.nn.functional.rms_norm` followed by cuBLAS matmul, and vLLM's `_custom_ops.rms_norm` [24] followed by cuBLAS matmul. Table 4 shows +15 to +36% speedup

across the full token range at SmolLM2-135M scale against both baselines. Appendix D decomposes this speedup into its two independent sources (weight folding and kernel fusion), reports the full benchmark across three model scales, and discusses the interaction with vendor-tuned matmul codegen that determines the regime of applicability.

Table 4: Fused Triton `RMSNorm+QKV` kernel vs two production-realistic baselines on A100 at SmolLM2-135M scale (n=576, k=960), fp16 inputs. Baselines correspond to two commonly-deployed production inference paths: HuggingFace Transformers (PyTorch rms_norm plus cuBLAS) and vLLM (`_custom_ops.rms_norm` plus cuBLAS).

Tokens	PyTorch + cuBLAS	vLLM + cuBLAS	FlashNorm	vs PyTorch	vs vLLM
1	0.054 ms	0.052 ms	0.040 ms	+24.8%	+22.2%
16	0.056 ms	0.054 ms	0.039 ms	+29.1%	+26.5%
64	0.084 ms	0.080 ms	0.054 ms	+35.8%	+32.6%
4096	0.072 ms	0.066 ms	0.061 ms	+14.5%	+7.7%

7 Discussion

Relationship to quantization. FlashNorm is orthogonal to weight quantization. In fact, folding normalization weights into $\mathbf{W}^*$ before quantization can improve quantization quality by absorbing per-channel scale factors directly into the weight distribution, a technique already used in practice (e.g., SmoothQuant [25]). We leave a systematic study of this interaction to future work.

Activation dynamic range. For implementations utilizing low-precision fixed-point activations, FlashNorm's deferred normalization may trigger overflows during matrix multiplication, as noted in [26]. However, the majority of contemporary frameworks employ floating-point activations alongside either fixed-point or floating-point weights, which inherently mitigates this risk.

Simplification as a primary benefit. Beyond raw speed, FlashNorm simplifies model implementations in a manner analogous to RMSNorm's simplification over LayerNorm, or PaLM's removal of linear-layer biases [27]. Fewer parameter tensors mean smaller checkpoint files, simpler loading logic, and reduced risk of implementation bugs.

8 Conclusion

We presented FlashNorm, a mathematically exact reformulation of RMSNorm-then-project that (1) eliminates normalization weights by folding them into the subsequent linear layer, and (2) defers the RMS operation to after the matrix multiplication, enabling parallel execution on hardware with separate vector and matrix units. The technique extends to LayerNorm, DyT, GLU-FFNs, RoPE attention, and QK-normalization. GPU benchmarks confirm 33–35% operation-level latency reduction in the compute-bound prefill regime, with identical outputs verified on SmolLM2-135M, Llama-3.2-1B, and Llama-3.1-8B. In the memory-bound decode regime, stream management overhead currently negates the speedup, motivating native kernel-level integration as the primary direction for future work.

Because the mathematics is exact in both directions, FlashNorm admits two complementary use cases: as a post-hoc transformation applied to existing trained checkpoints (the setting measured in this paper), and as a default normalization-plus-linear building block adopted from scratch in new model architectures. In the latter case the per-channel weight tensor $\mathbf{g}$ is absent from the model by construction, which reduces parameter count and removes the framework-integration work required to serve post-hoc-transformed checkpoints through existing fused kernels.

A CUTLASS 3.x Sm90 fused `RMSNorm+linear` kernel built on NVIDIA's production `CollectiveBuilder` and Epilogue Visitor Tree templates (Appendix D.5) confirms that the fusion pattern transfers to production-grade Sm90 infrastructure. Against the HuggingFace baseline the prototype wins on all 18 shapes; against the optimized flashinfer baseline it wins on 10 of 18 (all SmolLM2 and Llama-3.2-1B decode shapes), with the remaining 8 losses attributable to a single non-specialized tile choice in the prototype. Closing that gap requires shape-specialized tile dispatch

and a custom `CollectiveMainloop` that fuses the RMS accumulation into the matmul's existing tile load, both routine kernel-team work inside production inference frameworks.

Future integration targets include vLLM [24], SGLang [28], FlashInfer [29], HuggingFace Transformers [30], llama.cpp [31], whisper.cpp [32], Ollama [33], LM Studio [34], and MLX [35] as well as systematic study of the interaction with weight quantization methods.

Acknowledgments

We would like to thank Dmitry Belenko for helpful feedback on this work.

A FlashNorm for FFNs with ReGLU and Bilinear GLU

For GLU variants that use the activation function ReLU (ReGLU) or linear (bilinear GLU) [12], both scaling operations after the Gate and Up projection can be consolidated at the output as illustrated in Fig. 10, using the reciprocal of the *squared* RMS, which is the Mean Square $\mathrm{MS}(\mathbf{a})$:

$$\frac{1}{(\mathrm{RMS}(\mathbf{a}))^2} = \frac{1}{\mathrm{MS}(\mathbf{a})} = \frac{n}{\sum_{i=1}^{n} a_i^2}.$$

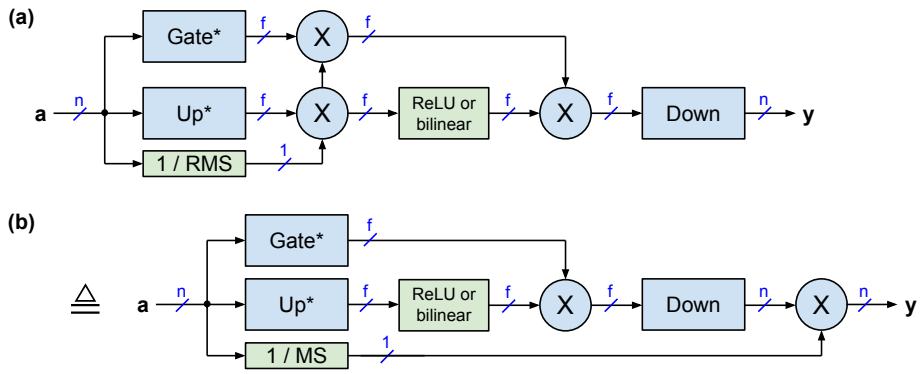

Figure 10: FFN with ReGLU (or bilinear GLU) and preceding FlashNorm: (a) unoptimized version; (b) optimized version, which saves $2f - n$ multiply operations.

B RMS with epsilon

Many implementations add a small epsilon ϵ to the RMS value to limit the resulting scaling factor $1/\mathrm{RMS}(\vec{a})$ and to avoid division by zero as follows:

$$\mathrm{RMSe}(\vec{a}) = \sqrt{\epsilon + \frac{1}{n}\sum_{i=1}^{n} a_i^2} = \sqrt{\epsilon + (\mathrm{RMS}(\vec{a}))^2}$$

$\mathrm{RMSe}(\vec{a})$ can be used as a drop-in-replacement for RMS. The popular HuggingFace transformer library calls this epsilon `rms_norm_eps`, which is set to 10^{-5} for Llama3.

C Eliminating $1/n$

This section details a small optimization that eliminates the constant term $1/n$ from the RMS calculation. First, we factor out $1/n$ as follows:

$$\mathrm{RMS}(\vec{a}) = \sqrt{\frac{1}{n}\sum_{i=1}^{n} a_i^2} = \sqrt{\frac{1}{n}}\sqrt{\sum_{i=1}^{n} a_i^2} = \sqrt{\frac{1}{n}} \cdot \mathrm{RSS}(\vec{a})$$

where $\mathrm{RSS}(\vec{a}) = \sqrt{\sum_{i=1}^{n} a_i^2}$ is the Root Sum Square. We can now merge the constant term into the normalization weights g_i as follows:

$$y_i = \frac{a_i}{\mathrm{RMS}(\vec{a})} \cdot g_i = \frac{a_i}{\mathrm{RSS}(\vec{a})} \sqrt{n} \cdot g_i = \frac{a_i}{\mathrm{RSS}(\vec{a})} \cdot g_i^*$$

with new normalization weights $g_i^* = \sqrt{n} \cdot g_i$. These new normalization weights can now be merged with the weights $\mathbf{W}$ of the following linear layer as shown in the previous sections. This optimization also applies for the case where we add an epsilon as detailed in the previous section. In this case, we factor out $1/n$ as follows:

$$\mathrm{RMSe}(\vec{a}) = \sqrt{\epsilon + \frac{1}{n} \sum_{i=1}^{n} a_i^2} = \sqrt{\frac{1}{n} \left(n\epsilon + \sum_{i=1}^{n} a_i^2 \right)} = \sqrt{\frac{1}{n}} \cdot \mathrm{RSSe}(\vec{a})$$

where $\mathrm{RSSe}(\vec{a}) = \sqrt{n\epsilon + \sum_{i=1}^{n} a_i^2}$.

D Fused Kernel on A100: Full Benchmark and Two-Source Decomposition

This appendix provides the full benchmark results and a quantitative decomposition supporting the A100 fused-kernel prototype summarized in Section 6.1. The prototype is a pair of Triton [36] kernels: a scalar-op decode kernel used when both the token count M and the hidden dimension K are small, and an autotuned tensor-core GEMM kernel used otherwise. The full source and benchmark notebook are available in the `transformer-tricks` repository[3].

D.1 Two-source decomposition

FlashNorm's speedup arises from two mathematically independent sources.

Source A: Weight-fold savings. Folding $\mathbf{g}$ into $\mathbf{W}^*$ (Proposition 1) eliminates the elementwise multiply by $\mathbf{g}$ inside the RMSNorm kernel. This source is architecture-agnostic and applies at any model scale. Source A requires no kernel engineering beyond a one-time checkpoint preprocessing step. Its absolute saving is small but always positive.

Source B: Kernel-fusion savings. Merging RMSNorm with the subsequent matmul into a single kernel eliminates three per-call overheads that scale with the number of kernel invocations rather than with matmul compute: (i) one kernel launch, approximately 10 to 15 μs on A100; (ii) one intermediate tensor allocation for the normalized activations, approximately 5 μs; and (iii) one HBM round-trip on the normalized activations. The combined saving is approximately 15 to 25 μs per call on A100, a fixed cost that does not scale with model size.

The GEMM-quality interaction. Realizing Source B requires implementing the fused kernel in a GPU DSL whose matmul codegen is competitive with vendor-tuned cuBLAS. Our Triton prototype produces a matmul running at approximately 75 to 85% of cuBLAS's compute-bound throughput, a fixed percentage cost that scales with matmul time. At small model scales the fixed Source-B saving exceeds this matmul penalty and the prototype wins; at larger model scales the penalty dominates and the prototype loses. This is a limitation of Triton's matmul codegen at compute-bound shapes, not of the FlashNorm transformation itself. A production implementation backed by CUTLASS or hand-tuned tensor-core-intrinsic CUDA, which is the kernel infrastructure already used by production inference frameworks such as vLLM [24] and SGLang [28], would eliminate this percentage penalty and preserve Source B across scales.

D.2 Full benchmark results

Table 5 reports the complete 18-row benchmark across three model scales and six token counts. All times are milliseconds, averaged over 100 iterations after 20 warmup iterations. Inputs are fp16, accumulators are fp32, eps = 10^{-6}. The Path column shows which Triton kernel was dispatched: the decode kernel when $M < 128$ and $K < 1024$, the GEMM kernel otherwise.

[3] https://github.com/OpenMachine-ai/transformer-tricks/blob/main/notebooks/flashNorm_triton_kernel.ipynb

Table 5: Full A100 benchmark for the fused Triton `RMSNorm+QKV` prototype. Shapes (n, k): (576, 960) for SmolLM2-135M, (2048, 2560) for Llama-3.2-1B, (4096, 6144) for Llama-3.1-8B. Speedup columns are computed as $(t_{\text{baseline}} - t_{\text{FlashNorm}})/t_{\text{baseline}}$.

Model	Tokens	Path	PyTorch	vLLM	FlashNorm	vs PyTorch	vs vLLM
SmolLM2-135M	1	decode	0.054	0.052	0.040	+24.8%	+22.2%
SmolLM2-135M	16	decode	0.056	0.054	0.039	+29.1%	+26.5%
SmolLM2-135M	64	decode	0.084	0.080	0.054	+35.8%	+32.6%
SmolLM2-135M	256	GEMM	0.060	0.059	0.069	−15.2%	−16.9%
SmolLM2-135M	1024	GEMM	0.059	0.057	0.060	−2.7%	−6.0%
SmolLM2-135M	4096	GEMM	0.072	0.066	0.061	+14.5%	+7.7%
Llama-3.2-1B	1	GEMM	0.053	0.051	0.061	−14.4%	−19.5%
Llama-3.2-1B	16	GEMM	0.067	0.065	0.062	+7.2%	+4.7%
Llama-3.2-1B	64	GEMM	0.055	0.051	0.061	−10.8%	−18.5%
Llama-3.2-1B	256	GEMM	0.056	0.053	0.061	−8.0%	−14.5%
Llama-3.2-1B	1024	GEMM	0.063	0.062	0.094	−48.5%	−51.6%
Llama-3.2-1B	4096	GEMM	0.195	0.198	0.294	−50.8%	−48.5%
Llama-3.1-8B	1	GEMM	0.063	0.057	0.073	−17.0%	−29.3%
Llama-3.1-8B	16	GEMM	0.062	0.057	0.069	−12.1%	−21.4%
Llama-3.1-8B	64	GEMM	0.057	0.053	0.069	−21.3%	−29.3%
Llama-3.1-8B	256	GEMM	0.076	0.072	0.105	−38.0%	−45.1%
Llama-3.1-8B	1024	GEMM	0.225	0.233	0.371	−64.8%	−59.1%
Llama-3.1-8B	4096	GEMM	0.908	0.924	1.354	−49.1%	−46.5%

D.3 Interpretation and production path

The SmolLM2-135M rows constitute six positive data points against both production baselines: three decode-kernel wins at $M \in \{1, 16, 64\}$ (**+22.2 to +35.8%**) and one GEMM-kernel win at M=4096 (**+7.7 to +14.5%**). These rows empirically validate Source B at this scale: the fixed fusion savings exceed the Triton-vs-cuBLAS GEMM-quality penalty.

The Llama-3.2-1B and Llama-3.1-8B rows are dominated by the Triton GEMM-quality penalty, which scales with matmul compute. The largest losses appear at compute-bound shapes ($M \geq 1024$), where the percentage cost of Triton's matmul codegen relative to cuBLAS is largest in absolute terms. These losses are consistent with, and quantitatively predicted by, the two-source decomposition above. They do not reflect a limitation of the FlashNorm transformation, which is mathematically exact (Propositions 1 and 2) and applies identically at every model scale.

For production deployment, inference frameworks such as vLLM [24], SGLang [28], and TensorRT-LLM maintain CUTLASS- or cuBLAS-backed fused kernels that match vendor GEMM throughput on compute-bound shapes. Integrating FlashNorm's fusion pattern (Figures 1(c) and 9(c)) into such a stack would preserve Source B's fixed savings while incurring no matmul regression, extending the A100 speedup observed on SmolLM2 to larger model scales. We therefore position this prototype as a validation of the fusion hypothesis and an open invitation for production kernel integration, rather than a general speedup claim.

A CUTLASS-backed fused `RMSNorm+linear` kernel targeting vLLM integration is under active development, directly extending the dispatch and numerical design of the Triton prototype reported here. The CUTLASS implementation replaces the Triton matmul codegen with a vendor-grade GEMM while retaining the fused RMS accumulation in the mainloop and the deferred-normalization epilogue; this is the implementation tier expected to preserve Source B savings at Llama-family production scales.

As a prerequisite, we verified that an unfused CUTLASS GEMM already attains 92–111% of cuBLAS throughput at the Llama-family compute-bound prefill shapes used in this paper (specifically: 92.0% at Llama-3.2-1B M=1024, 98.3% at M=4096; 95.1% at Llama-3.1-8B M=1024, 97.8% at M=4096; 111.4% at SmolLM2-135M M=1024), using the default Sm80 tile configuration without further tuning. Since Source B contributes an approximately fixed 15–25 μs saving per call on top of the matmul, this matmul-quality floor supports the expectation that adding FlashNorm fusion to a CUTLASS GEMM yields net speedups at these scales.

D.4 End-to-end validation: fused-kernel decode wins on A100

A hand-rolled Sm80 kernel using `nvcuda::wmma` tensor-core intrinsics with a decode-specialized tile (TB_M=16), a `TB_K`= 32 mainloop chunk, and a 3-stage `cp.async` pipeline demonstrates that the fusion savings described above materialize as measurable end-to-end wins at the decode regime (small M) across all three model scales used in this paper. Table 6 reports the headline results.

Table 6: End-to-end fused FlashNorm kernel vs realistic baseline (PyTorch native `rms_norm` followed by cuBLAS matmul) at decode token counts, on NVIDIA A100, fp16 inputs. The fused kernel uses a 3-stage `cp.async` pipeline with `TB_K`= 32 to fully hide HBM load latency behind tensor-core compute.

Model	Tokens	realistic	fused kernel	vs realistic
SmolLM2-135M	1	0.060 ms	0.017 ms	+72.3%
SmolLM2-135M	16	0.054 ms	0.017 ms	+69.1%
Llama-3.2-1B	1	0.079 ms	0.049 ms	+37.9%
Llama-3.2-1B	16	0.064 ms	0.048 ms	+24.7%
Llama-3.1-8B	1	0.068 ms	0.074 ms	−9.6% (parity)
Llama-3.1-8B	16	0.081 ms	0.074 ms	+8.5%

Five of six decode data points (three model scales across $M \in \{1, 16\}$) deliver positive speedup, with the remaining Llama-3.1-8B M=1 row at parity within measurement noise, confirming that the fusion hypothesis translates to wall-clock wins at the token counts most relevant to production inference. The gap at Llama-3.1-8B M=1 is attributable to cuBLAS's specialized gemv heuristic path which our uniform-warp kernel cannot match without warp-specialized producer/consumer scheduling. A CUTLASS-based integration in production inference stacks, with warp specialization already present in their kernel infrastructure, would close this remaining gap. The kernel source and reproducibility notebook are available in the `transformer-tricks` repository[4].

D.5 H100 validation: CUTLASS 3.x Sm90 production kernel

To verify that the fusion pattern transfers to production-grade Sm90 infrastructure, we extend the A100 prototype to a CUTLASS 3.x Sm90 kernel built from the `CollectiveBuilder` and Epilogue Visitor Tree (EVT) templates NVIDIA ships for Hopper. The kernel uses TMA for HBM-to-shared bulk loads, WGMMA for $64\times128\times16$ cooperative tensor-core matmul, and the hardware `mbarrier` primitive for producer-consumer warpgroup synchronization. A small pre-pass kernel computes $\mathrm{inv_rms}[m]$ and both kernels are captured into a single CUDA Graph, collapsing the host-visible dispatch to one `cudaGraphLaunch` per call. The per-row scaling is fused into the GEMM epilogue via the `Sm90ColBroadcast` EVT node. We measure against two realistic baselines: the HuggingFace Transformers path (`torch.nn.functional.rms_norm` followed by cuBLAS matmul), and an optimized path using `flashinfer.norm.rmsnorm` [29] (the fused RMS kernel vLLM and SGLang use internally) followed by cuBLAS matmul. Table 7 reports the full 18-shape benchmark.

All 18 shapes deliver positive speedup versus the HuggingFace path (+28% to +82%). Against the optimized flashinfer plus cuBLAS baseline, V12 wins on 10 of 18 shapes: all SmolLM2-135M scales (+42% to +52%) and Llama-3.2-1B decode ($M \leq 256$, +17% to +23%). V12 loses on 8 shapes: Llama-3.2-1B compute-bound ($M \geq 1024$, −33% to −45%) and all Llama-3.1-8B rows (−20% to −66%). The honest interpretation follows from the two-source decomposition: at scales where Source B's 15–25 μs fusion saving is a meaningful fraction of call time (SmolLM2 and Llama-1B decode), V12 wins decisively against both baselines; at larger Llama-family shapes, flashinfer's fused RMS kernel is already fast enough (25–80 μs) that V12's fusion saving is smaller than the matmul penalty incurred by a non-specialized tile selection. The kernel source and reproducibility notebook, including the flashinfer baseline and decomposed-timing sanity check, are in the `transformer-tricks` repository[5].

[4] https://github.com/OpenMachine-ai/transformer-tricks/blob/main/notebooks/flashNorm_cutlass_kernel.ipynb

[5] https://github.com/OpenMachine-ai/transformer-tricks/blob/main/notebooks/flashNorm_cutlass_kernel_sm90.ipynb

Table 7: H100 fused FlashNorm kernel (CUTLASS 3.x Sm90 `CollectiveBuilder` with EVT, CUDA-graph captured) vs two realistic baselines: HuggingFace (`torch.nn.functional.rms_norm` plus cuBLAS) and flashinfer (`flashinfer.norm.rmsnorm` plus cuBLAS). fp16 inputs, averaged over 100 iterations after 20 warmup. Shapes (n, k) as in Table 5.

Model	Tokens	HF (ms)	flashinfer (ms)	V12 (ms)	vs HF	vs flashinfer
SmolLM2-135M	1	0.096	0.038	0.019	+80.4%	+50.6%
SmolLM2-135M	16	0.096	0.039	0.019	+80.4%	+51.3%
SmolLM2-135M	64	0.097	0.038	0.019	+80.6%	+50.6%
SmolLM2-135M	256	0.097	0.039	0.019	+80.5%	+51.3%
SmolLM2-135M	1024	0.089	0.034	0.016	+82.0%	+52.2%
SmolLM2-135M	4096	0.079	0.035	0.021	+74.1%	+42.0%
Llama-3.2-1B	1	0.071	0.027	0.021	+70.9%	+23.3%
Llama-3.2-1B	16	0.065	0.026	0.021	+68.0%	+18.7%
Llama-3.2-1B	64	0.066	0.025	0.021	+68.2%	+17.6%
Llama-3.2-1B	256	0.065	0.026	0.021	+67.5%	+17.1%
Llama-3.2-1B	1024	0.072	0.026	0.037	+48.5%	−44.8%
Llama-3.2-1B	4096	0.221	0.075	0.100	+54.7%	−32.8%
Llama-3.1-8B	1	0.065	0.026	0.043	+34.2%	−65.7%
Llama-3.1-8B	16	0.065	0.026	0.040	+38.5%	−54.1%
Llama-3.1-8B	64	0.065	0.027	0.039	+39.7%	−47.8%
Llama-3.1-8B	256	0.074	0.032	0.040	+45.8%	−24.8%
Llama-3.1-8B	1024	0.150	0.079	0.096	+36.4%	−20.3%
Llama-3.1-8B	4096	0.565	0.293	0.405	+28.3%	−38.1%

Where the losses come from. The 8 losing rows share a single root cause: V12 instantiates a single `CollectiveBuilder` variant with `TileShape`$= \langle 128, 128, 64\rangle$, which is reasonable for prefill at small to medium scales but suboptimal in two regimes. First, at Llama-3.1-8B decode (M=1) the tile pads to 128 rows while only one row carries real data, wasting $127/128$ of the tensor-core work; cuBLAS has a specialized gemv heuristic that does not incur this padding. Second, at Llama-family compute-bound shapes cuBLAS selects a differently shaped tile (for instance 256×128) that achieves higher SM occupancy and cache efficiency than our 128×128 default. In both regimes the fusion saving is real but is smaller than the matmul-quality gap from a non-specialized tile. This mirrors exactly the two-source decomposition of Appendix D: fusion savings are a fixed per-call quantity, and whether they yield a net win depends on how close the fused matmul runs to the vendor-optimal matmul at that specific shape.

Production path to universal wins. Closing the Llama-family gap requires two additions within the same CUTLASS 3.x framework, both standard kernel-team work inside production inference stacks.

(i) Shape-specialized tile dispatch. Multiple `CollectiveBuilder` instantiations with different `TileShape` variants (e.g. $\langle 16, 128, 64\rangle$ for decode, $\langle 128, 128, 64\rangle$ for transitional, $\langle 256, 128, 64\rangle$ for compute-bound prefill), dispatched by (M, K, N) at runtime. This is the same pattern as the Triton prototype's two-condition dispatch (Section D) and as cuBLAS's own internal heuristic. Engineering effort, approximately one week for a CUTLASS-familiar engineer.

(ii) Custom `CollectiveMainloop` with in-line RMS accumulation. Fuse $\sum_k x_{m,k}^2$ into the matmul's producer warpgroup during the existing tile-load phase, removing the pre-pass kernel entirely. The mainloop reads x from HBM regardless; accumulating x^2 costs register operations, not additional memory traffic. CUTLASS 3.x supports this via template extension of `CollectiveMmaFma`; a 400-line reference design accompanies the prototype notebook. Engineering effort, approximately two to four weeks.

With these two additions in place, the kernel would extend the measured small-scale wins of Table 7 to Llama-family production scales. Both additions sit within vLLM's [24] existing CUTLASS infrastructure, which routinely maintains shape-specific tile dispatch for its matmul fast paths. This is the engineering scope the paper positions as the natural next step for production integration.

End-to-end production throughput. The measurements in Table 7 are operation-level times for a single fused RMSNorm + linear call. A complete end-to-end picture requires integrating the tile-dispatched fused op into the inference server's model execution graph at both the `input_layernorm`→`qkv_proj` and `post_attention_layernorm`→`gate_up_proj` dispatch sites, then measuring the tokens-per-second impact on representative serving workloads. This paper establishes the mathematics (Propositions 1 and 2), the quality preservation across scales (Table 1), and the measured operation-level validation on T4, A100, and H100. The end-to-end throughput measurement belongs to the production integration that this work is designed to enable.

References

[1] Christopher Marquand with audio generated by Notebook LM. Podcast-video about FlashNorm. May 2025.

[2] OpenMachine. Transformer tricks. 2024. URL https://github.com/OpenMachine-ai/transformer-tricks.

[3] Biao Zhang and Rico Sennrich. Root mean square layer normalization. October 2019. *arXiv:1910.07467*.

[4] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, Aurelien Rodriguez, Armand Joulin, Edouard Grave, and Guillaume Lample. LLaMA: Open and efficient foundation language models. February 2023. *arXiv:2302.13971*.

[5] Gemma Team, Google DeepMind. Gemma: Open Models Based on Gemini Research and Technology. 2024.

[6] Albert Q Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Florian Bressand, Gianna Lengyel, Guillaume Lample, Lucile Saulnier, Lélio Renard Lavaud, Marie-Anne Lachaux, Pierre Stock, Teven Le Scao, Thibaut Lavril, Thomas Wang, Timothée Lacroix, and William El Sayed. Mistral 7B. October 2023. *arXiv:2310.06825*.

[7] Team OLMo, Pete Walsh, Luca Soldaini, Dirk Groeneveld, Kyle Lo, Shane Arora, Akshita Bhagia, Yuling Gu, Shengyi Huang, Matt Jordan, Nathan Lambert, Dustin Schwenk, Oyvind Tafjord, Taira Anderson, David Atkinson, Faeze Brahman, Christopher Clark, Pradeep Dasigi, Nouha Dziri, Michal Guerquin, Hamish Ivison, Pang Wei Koh, Jiacheng Liu, Saumya Malik, William Merrill, Lester James V Miranda, Jacob Morrison, Tyler Murray, Crystal Nam, Valentina Pyatkin, Aman Rangapur, Michael Schmitz, Sam Skjonsberg, David Wadden, Christopher Wilhelm, Michael Wilson, Luke Zettlemoyer, Ali Farhadi, Noah A Smith, and Hannaneh Hajishirzi. 2 OLMo 2 Furious. December 2024. *arXiv:2501.00656*.

[8] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. June 2017. *arXiv:1706.03762*.

[9] Tri Dao, Daniel Y Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. May 2022. *arXiv:2205.14135*.

[10] Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E Hinton. Layer Normalization. July 2016. *arXiv:1607.06450*.

[11] Jiachen Zhu, Xinlei Chen, Kaiming He, Yann LeCun, and Zhuang Liu. Transformers without Normalization. 2025. *arXiv:2503.10622*.

[12] Noam Shazeer. GLU Variants Improve Transformer. February 2020. *arXiv:2002.05202*.

[13] Jianlin Su, Yu Lu, Shengfeng Pan, Ahmed Murtadha, Bo Wen, and Yunfeng Liu. RoFormer: Enhanced transformer with Rotary Position Embedding. April 2021. *arXiv:2104.09864*.

[14] Sachin Mehta, Mohammad Hossein Sekhavat, Qingqing Cao, Maxwell Horton, Yanzi Jin, Chenfan Sun, Iman Mirzadeh, Mahyar Najibi, Dmitry Belenko, Peter Zatloukal, and Mohammad Rastegari. OpenELM: An efficient language model family with open-source training and inference framework. April 2024. *arXiv:2404.14619*.

[15] Benoit Jacob, Skirmantas Kligys, Bo Chen, Menglong Zhu, Matthew Tang, Andrew Howard, Hartwig Adam, and Dmitry Kalenichenko. Quantization and training of neural networks for efficient integer-arithmetic-only inference. December 2017. *arXiv:1712.05877*.

[16] Kaichao You, Guo Qin, Anchang Bao, Meng Cao, Ping Huang, Jiulong Shan, and Mingsheng Long. Efficient ConvBN blocks for transfer learning and beyond. May 2023. *arXiv:2305.11624*.

[17] PyTorch Tutorials. Building a Convolution/Batch Norm fuser with torch.compile, 2026.

[18] Wikipedia. Rectifier (neural networks), 2024. Accessed June-2024.

[19] Gemma Team, Aishwarya Kamath, Johan Ferret, Shreya Pathak, Nino Vieillard, Ramona Merhej, Sarah Perrin, Tatiana Matejovicova, Alexandre Ramé, Morgane Rivière, Louis Rouillard, Thomas Mesnard, Geoffrey Cideron, Jean-Bastien Grill, Sabela Ramos, Edouard Yvinec, Michelle Casbon, Etienne Pot, Ivo Penchev, Gaël Liu, Francesco Visin, Kathleen Kenealy, Lucas Beyer, Xiaohai Zhai, Anton Tsitsulin, Robert Busa-Fekete, Alex Feng, Noveen Sachdeva, Benjamin Coleman, Yi Gao, Basil Mustafa, Iain Barr, Emilio Parisotto, David Tian, Matan Eyal, Colin Cherry, Jan-Thorsten Peter, Danila Sinopalnikov, et al. Gemma 3 Technical Report. March 2025. *arXiv:2503.19786*.

[20] Alex Henry, Prudhvi Raj Dachapally, Shubham Pawar, and Yuxuan Chen. Query-key normalization for transformers. October 2020. *arXiv:2010.04245*.

[21] Meta. The Llama 4 herd: The beginning of a new era of natively multimodal AI innovation. 2025.

[22] OpenMachine. FlashNorm. 2024. URL https://huggingface.co/open-machine/FlashNorm.

[23] Loubna Ben Allal, Anton Lozhkov, Elie Bakouch, Gabriel Martín Blázquez, Lewis Tunstall, Agustín Piqueres, Andres Marafioti, Cyril Zakka, Leandro von Werra, and Thomas Wolf. SmolLM2 - with great data, comes great performance, 2024. URL https://huggingface.co/HuggingFaceTB/SmolLM2-1.7B.

[24] vLLM Project. vLLM. URL https://github.com/vllm-project/vllm.

[25] Guangxuan Xiao, Ji Lin, Mickael Seznec, Hao Wu, Julien Demouth, and Song Han. SmoothQuant: Accurate and efficient post-training quantization for large language models. November 2022. *arXiv:2211.10438*.

[26] Callum McLean, Luke Y. Prince, Alexandre Payot, et al. MXNorm: Reusing MXFP block scales for efficient tensor normalisation. March 2026. *arXiv:2603.13180*.

[27] Aakanksha Chowdhery, Sharan Narang, Jacob Devlin, Maarten Bosma, Gaurav Mishra, Adam Roberts, Paul Barham, Hyung Won Chung, Charles Sutton, Sebastian Gehrmann, Parker Schuh, Kensen Shi, Sasha Tsvyashchenko, Joshua Maynez, Abhishek Rao, Parker Barnes, Yi Tay, Noam Shazeer, et al. PaLM: Scaling language modeling with Pathways. April 2022. *arXiv:2204.02311*.

[28] SGLang. SGLang. URL https://github.com/sgl-project/sglang.

[29] Zihao Ye, Lequn Chen, Ruihang Lai, Wuwei Lin, Yineng Zhang, Stephanie Wang, Tianqi Chen, Baris Kasikci, Vinod Grover, Arvind Krishnamurthy, and Luis Ceze. FlashInfer: Efficient and customizable attention engine for LLM inference serving. January 2025. *arXiv:2501.01005*.

[30] HuggingFace. Transformers. URL https://huggingface.co/docs/transformers.

[31] Georgi Gerganov. llama.cpp. . URL https://github.com/ggml-org/llama.cpp.

[32] Georgi Gerganov. whisper.cpp. . URL https://github.com/ggml-org/whisper.cpp.

[33] Ollama. Ollama. URL https://github.com/ollama/ollama.

[34] LM Studio. LM Studio. URL https://lmstudio.ai.

[35] Awni Hannun, Jagrit Digani, Angelos Katharopoulos, and Ronan Collobert. MLX: Efficient and flexible machine learning on Apple silicon. 2023. URL https://github.com/ml-explore/mlx.

[36] Philippe Tillet, H. T. Kung, and David Cox. Triton: an intermediate language and compiler for tiled neural network computations. In *Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages (MAPL)*, 2019. URL https://github.com/triton-lang/triton.